# Bash: моя проверка HTTPS API

Практика 10-12 октября 2026 / выполнено с подсказками

### Задача и результат

Я учился читать и писать Bash, а не только запускать готовые команды. В файле /home/ops/bash-01.sh я собрал проверку s12-api: состояние systemd, HTTPS с проверкой доверия сертификату, точный HTTP 200 и поля JSON status=ok, service=s12-api. Изменения выполнял сам, конструкции и исправления разбирал с наставником.

На финальном скриншоте 12 октября: сервис active, HTTP code: 200, правильный JSON, сообщение успешной проверки и код 0. Это доказательство конкретного запуска, а не непрерывной исправности сервера.

### Что я понял

Активный процесс не гарантирует правильный ответ приложения. curl может успешно получить HTTP-ответ, содержащий ошибку приложения. Поэтому я разделил проверку процесса, транспорта/HTTP и содержимого JSON.

stdout хранит обычный результат, stderr - диагностику. Код завершения независим от напечатанного текста: ERROR сам по себе не означает ненулевой код. Я подтвердил это экспериментом, затем добавил exit 0 и exit 1.

### Новые конструкции: данные и выполнение

```
service="${1:-s12-api}"
current_user="$(whoami)"
printf 'Testing service: %s\n' "$service"
```

${1:-s12-api} выбирает первый аргумент или значение по умолчанию, если аргумент отсутствует либо пустой. Это не список разрешённых сервисов. При запуске с ssh проверяется ssh.service в системе.

$(...) сохраняет stdout команды. $? содержит код последней команды, поэтому я сохраняю его сразу, до следующего printf. %s - место подстановки printf; $service - значение переменной. Опечатка "%service" печатала буквальный текст.

```
if command; then
    printf 'OK\n'
else
    printf 'ERROR\n' >&2
    exit 1
fi
```

if выбирает ветку по коду команды: 0 или ненулевой. exit завершает именно запущенный скрипт; ввод exit прямо в интерактивной оболочке завершает оболочку.

# Новые приёмы проверки

## Потоки, JSON и HTTP

### Потоки и конвейер

Перенаправление > /dev/null скрывает stdout; 2> /dev/null скрывает stderr. Я проверил оба варианта: скрытие текста не изменило код 1. Конвейер | передаёт stdout одной команды на stdin следующей.

```
if printf '%s\n' "$response" |
    jq -e '.status == "ok" and .service == "s12-api"' > /dev/null; then
    printf 'OK: API returned the expected JSON\n'
    exit 0
else
    printf 'ERROR: invalid JSON or unexpected field values\n' >&2
    exit 1
fi
```

jq разбирает структуру JSON. and требует обоих совпадений; регистр важен. -e делает результат фильтра доступным через код завершения. Для этого конвейера в обычном режиме Bash учитывает код последней команды, jq.

### Новые параметры уже знакомого curl

Полный разбор curl, сертификатов и --resolve уже есть в сетевом реферате; здесь я фиксирую только новые применения.

- --silent --show-error: без индикатора загрузки, но с диагностикой ошибок.
- --fail: HTTP 400 и выше приводит к ошибке curl; показанный HTTP 404 дал curl (22), мой скрипт вернул 1 через exit.
- --write-out '\n%{http_code}': добавляет HTTP-код после тела ответа. Одного --fail недостаточно для требования именно 200.

```
http_code="${response_with_code##*$'\n'}"
response="${response_with_code%$'\n'*}"
if [[ "$http_code" != "200" ]]; then
    printf 'ERROR: expected HTTP 200, received %s\n' "$http_code" >&2
    exit 1
fi
```

$'\n' обозначает перевод строки. ## удаляет самый длинный подходящий префикс, % - самый короткий подходящий суффикс. Поэтому я отделяю последнюю строку с HTTP-кодом от тела одного и того же запроса.

Пробел перед ]] обязателен. bash -n проверяет синтаксис без выполнения скрипта; он не обнаружит опечатку в имени переменной или неверное ожидаемое значение.

# Проверки и границы результата

## Что подтверждено, что пока не проверено

### Показанные тесты

- true/false: я получил обе ветки; после явного exit успех дал 0, ошибка 1.
- Несуществующее имя s12dasd: проверка systemd не подтвердила активность, скрипт вернул 1.
- Пустой аргумент: выбран s12-api; аргумент ssh: проверен ssh.service, результат 0.
- HTTP-путь bash-training-missing: active-процесс остался работать, сервер вернул 404, curl сообщил 22, скрипт вернул 1.
- Ожидаемое имя s12=api вместо s12-api: JSON не прошёл сравнение, код 1.
- Ожидаемый status=maintenance при фактическом ok: JSON не прошёл сравнение, код 1.
- Финальный положительный запуск: HTTP 200, корректные поля JSON, код 0.

### Ошибки, которые я разобрал

Я исправлял % вместо $ в аргументе printf, отсутствующий пробел перед ]], responde_with_code вместо response_with_code и OK вместо ok. После неоднозначной инструкции по замене блока потерялась ветка else для curl; наставник дал цельный блок, и я вернул обработку ошибки. Это работа с помощью, не независимая сдача задания.

### Ограничения учебного скрипта

HTTPS-адрес, CA и ожидаемое имя JSON фиксированы для s12-api. Аргумент меняет только проверяемую systemd-службу. Переменная port пока печатается, но URL и --resolve содержат 8443 буквально. Скрипт не является универсальным мониторингом.

Не показан отрицательный тест точного HTTP-кода на ответе 201/3xx после последней правки. Не показаны отдельные тесты тайм-аута, отказа TLS, недоступного порта, отсутствующего jq и повреждённого JSON. Старые отрицательные тесты не означают полный регресс после финального изменения.

В репозиторий добавлен отчёт, не выданная за оригинал реконструкция серверного файла: полное финальное содержимое скрипта после исправлений отдельно не выгружалось.

### Дальше

Я завершаю этот учебный блок и возвращаюсь к DevOps-роадмапу. Новые функции скрипта пока не добавляю. Числовые уровни самостоятельности не повышаю: для них нужна новая задача без заранее показанного решения.

Связанные материалы: network-09-12.md (HTTP/TLS), access-terminal-2026-10-12.md (доступ, оболочка и инцидент SSH). В следующих рефератах уже разобранные команды повторно не объясняю.